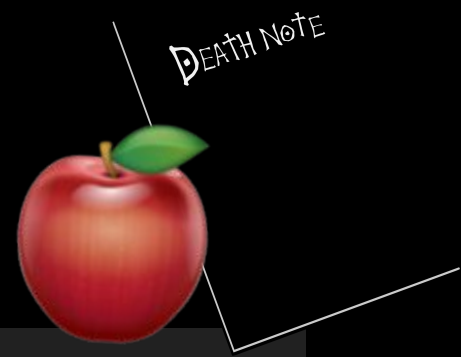


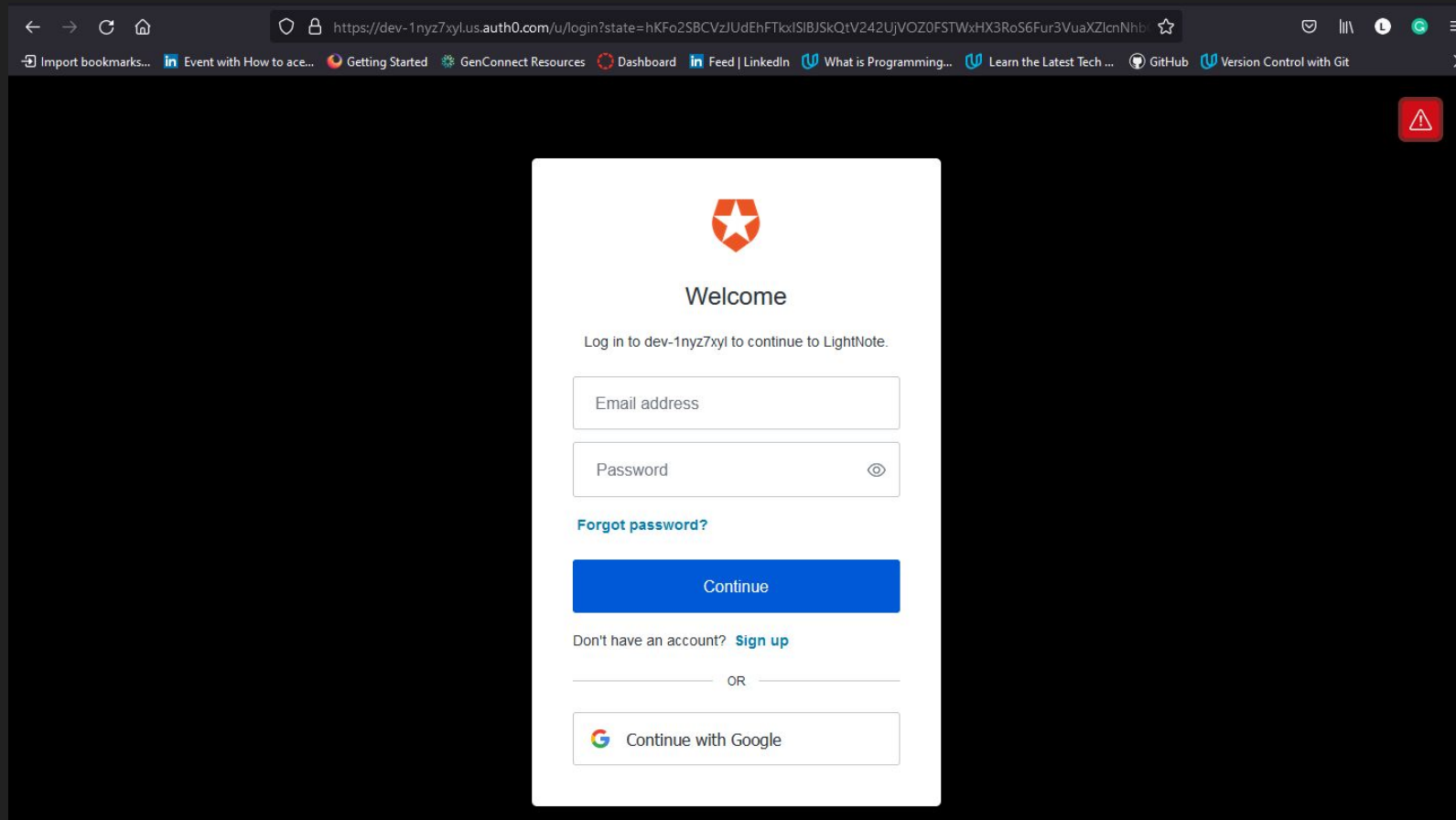


Light's Notebook

RESTful API using JAVA, Spring, Postman,
OAuth

Crystal, Nate, Michael & Lindsey

Logging in localhost:8080 redirects to



← → ↻ 🏠 <https://dev-1nyz7xyl.us.auth0.com/u/login?state=hKFo2SBCVzJUdEhFTkxISIBJSkQtV242UjVOZ0FSTWxHX3RoS6Fur3VuaXZlcnNhb> 📌 ⏏ 🔔 🌐 ☰

🔖 Import bookmarks... 🌐 Event with How to ace... 🚀 Getting Started 🌐 GenConnect Resources 🌐 Dashboard 🌐 Feed | LinkedIn 🌐 What is Programming... 🌐 Learn the Latest Tech ... 🌐 GitHub 🌐 Version Control with Git >





Welcome

Log in to dev-1nyz7xyl to continue to LightNote.



[Forgot password?](#)

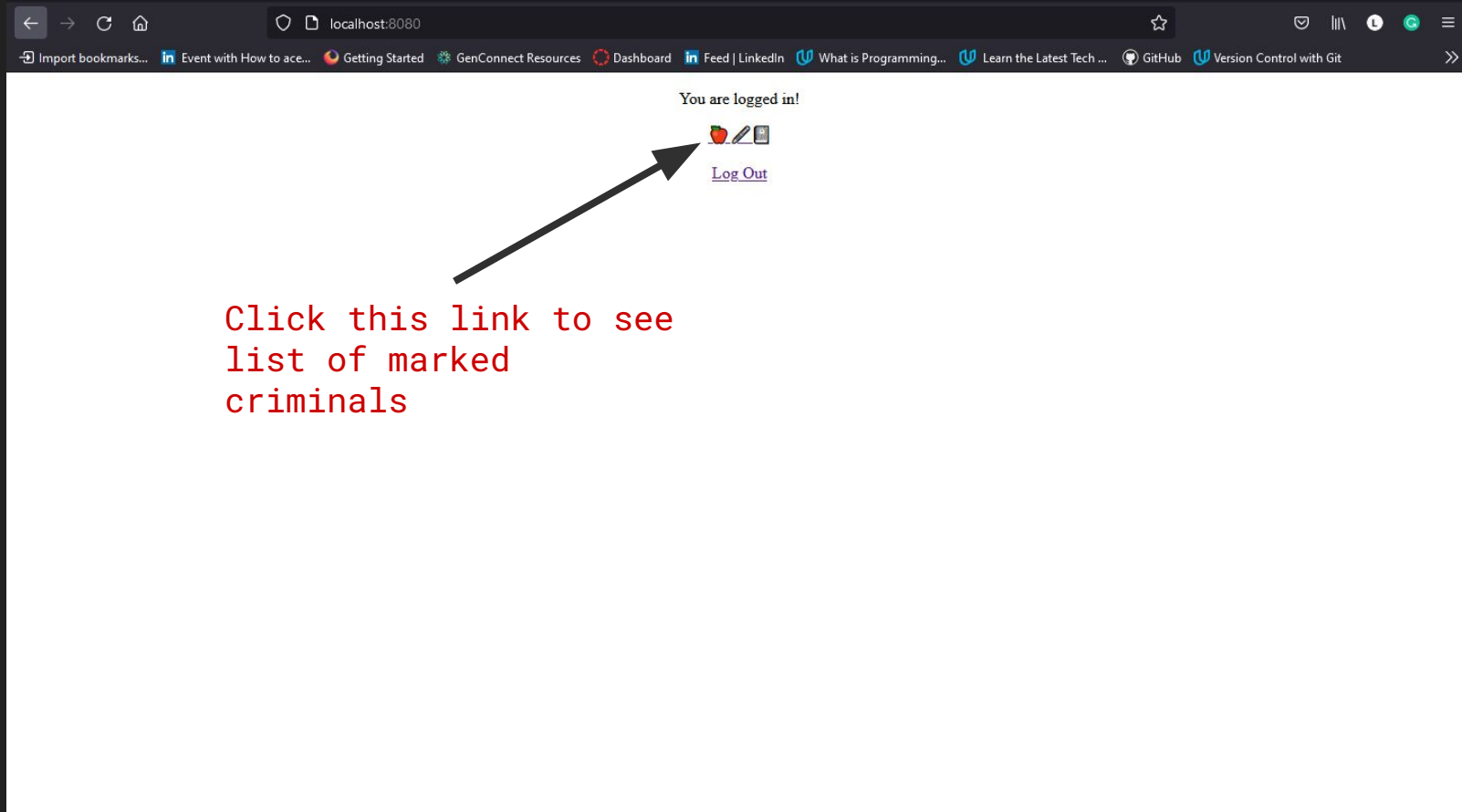
[Continue](#)

Don't have an account? [Sign up](#)

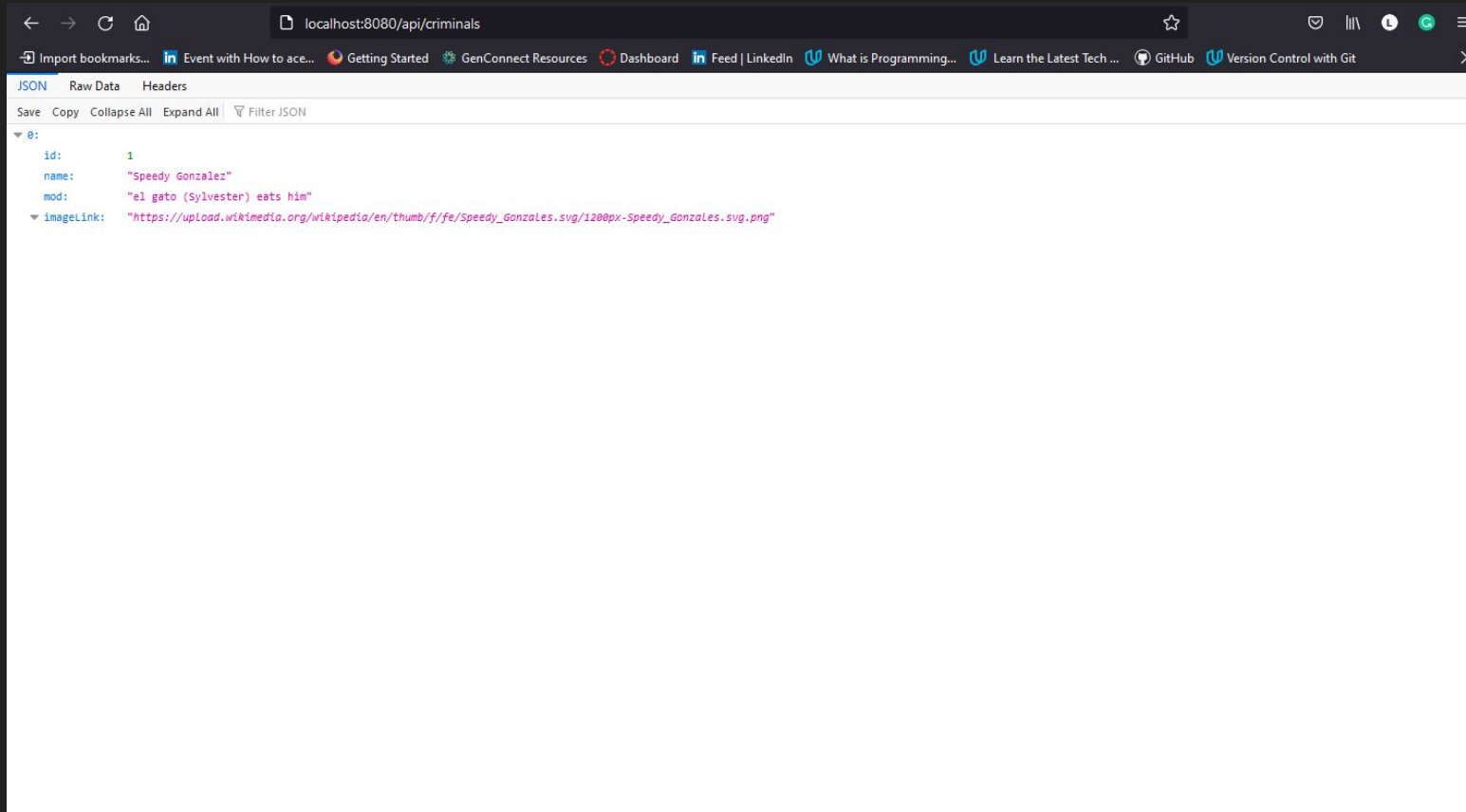
OR

 [Continue with Google](#)

Once authenticated you are logged in



Once authenticated you are logged in localhost:8080/criminals (GET)



<http://localhost:8080/api/criminals>

Adding a criminal to the list (POST)

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** localhost:8080/api/criminals/
- Body:** This request does not have a body
- Response:**
 - Status:** 200 OK (highlighted with a red circle)
 - Time:** 625 ms
 - Size:** 576 B
- Response Body (JSON):**

```
1 {
2   "id": 3,
3   "name": "Naoki Koro Kitayama",
4   "mod": "heart attack",
5   "imageLink": "http://austinkleon.com/wp-content/uploads/2008/02/stick_figure.gif"
6 }
7
8
```

http://localhost:8080/api/criminals/{id}

Editing a criminal (PUT)

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** localhost:8080/api/criminals/1
- Body:** A JSON object with the following fields:

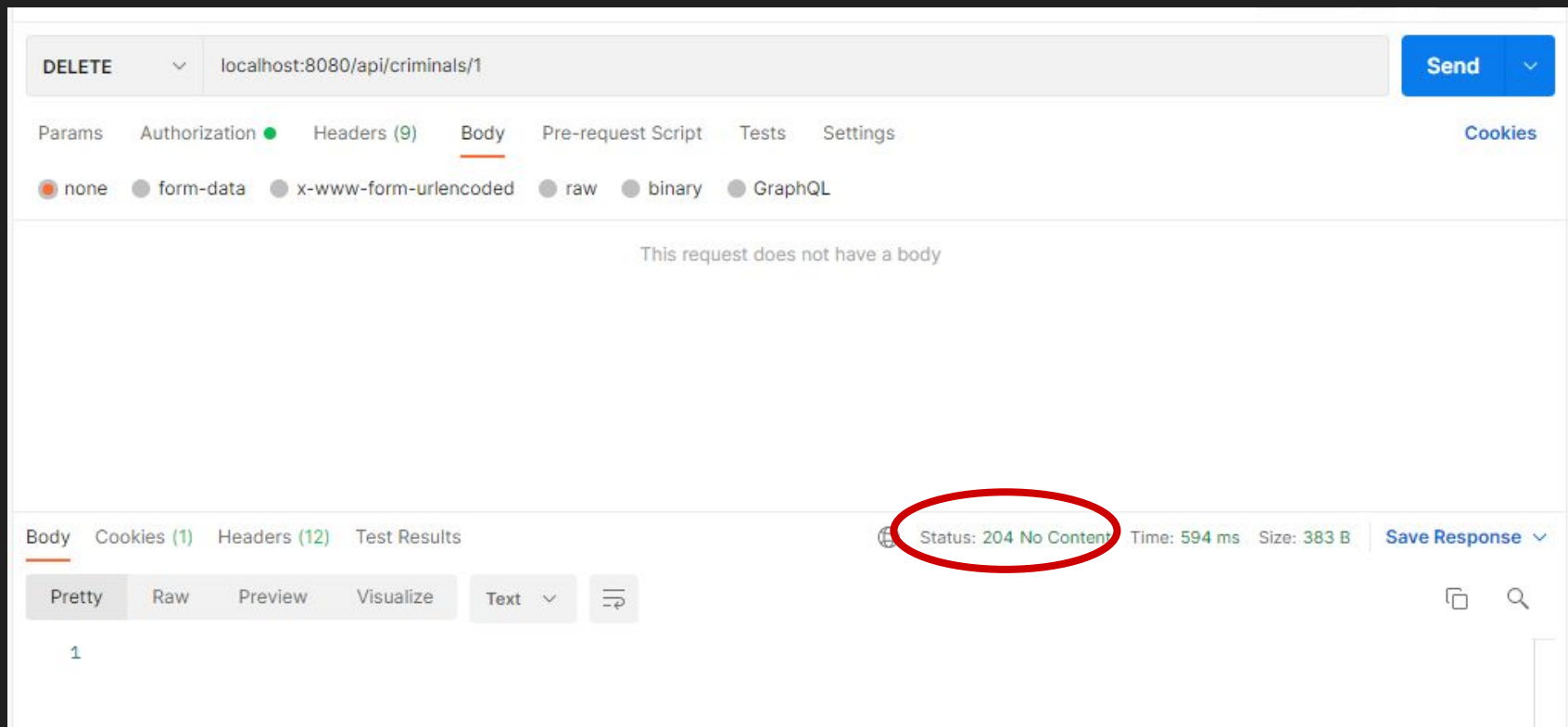
```
{  "name": "Speedy Gonzalez",  "mod": "cancel culture",  "imageLink": "https://upload.wikimedia.org/wikipedia/en/thumb/f/fe/Speedy_Gonzales.svg/1200px-Speedy_Gonzales.svg.png"}
```
- Status:** 200 OK (highlighted with a red circle)
- Time:** 182 ms
- Size:** 609 B
- Response Body:** A JSON object with the following fields:

```
{  "id": 1,  "name": "Speedy Gonzalez",  "mod": "cancel culture",  "imageLink": "https://upload.wikimedia.org/wikipedia/en/thumb/f/fe/Speedy_Gonzales.svg/1200px-Speedy_Gonzales.svg.png"}
```

<http://localhost:8080/api/criminals/{id}>

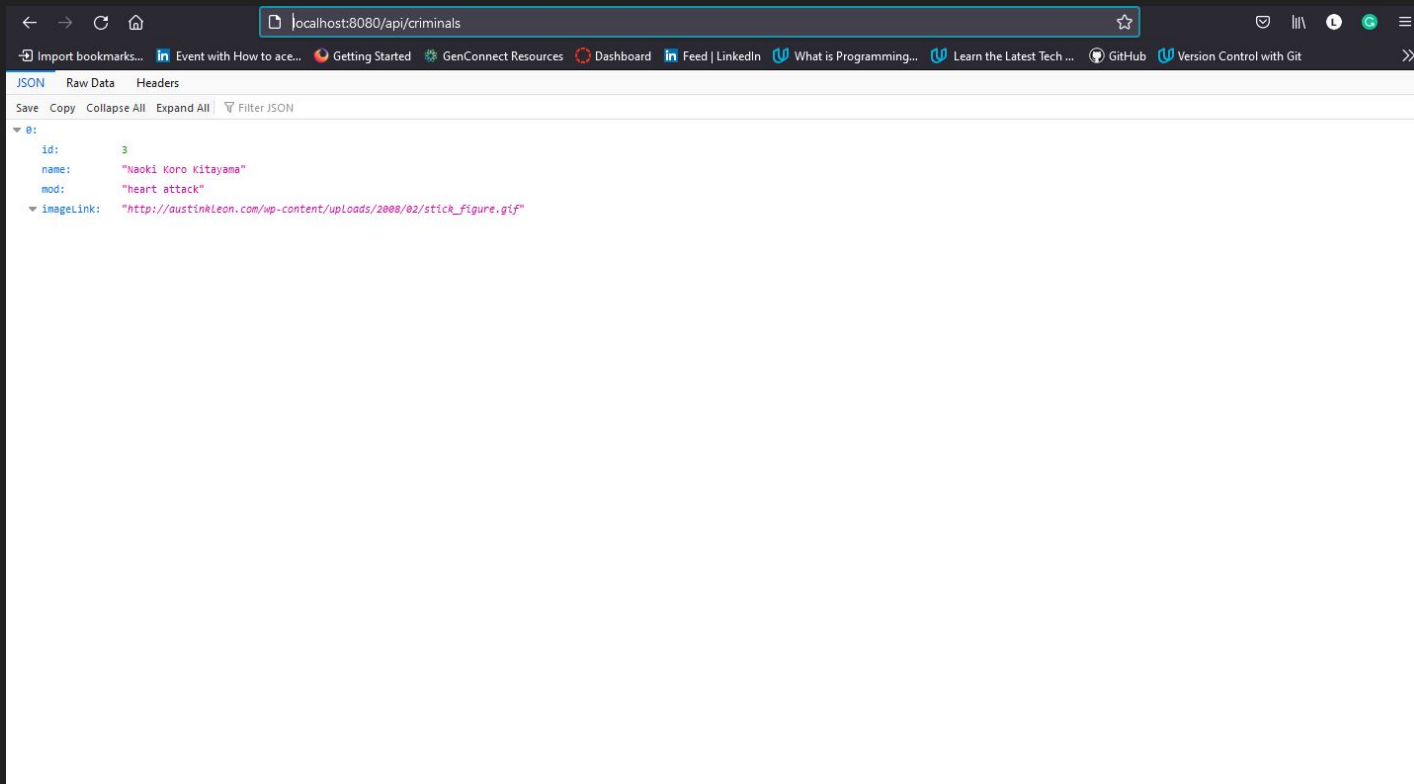
Deleting a criminal (DELETE)

Speedy is criminal id 1

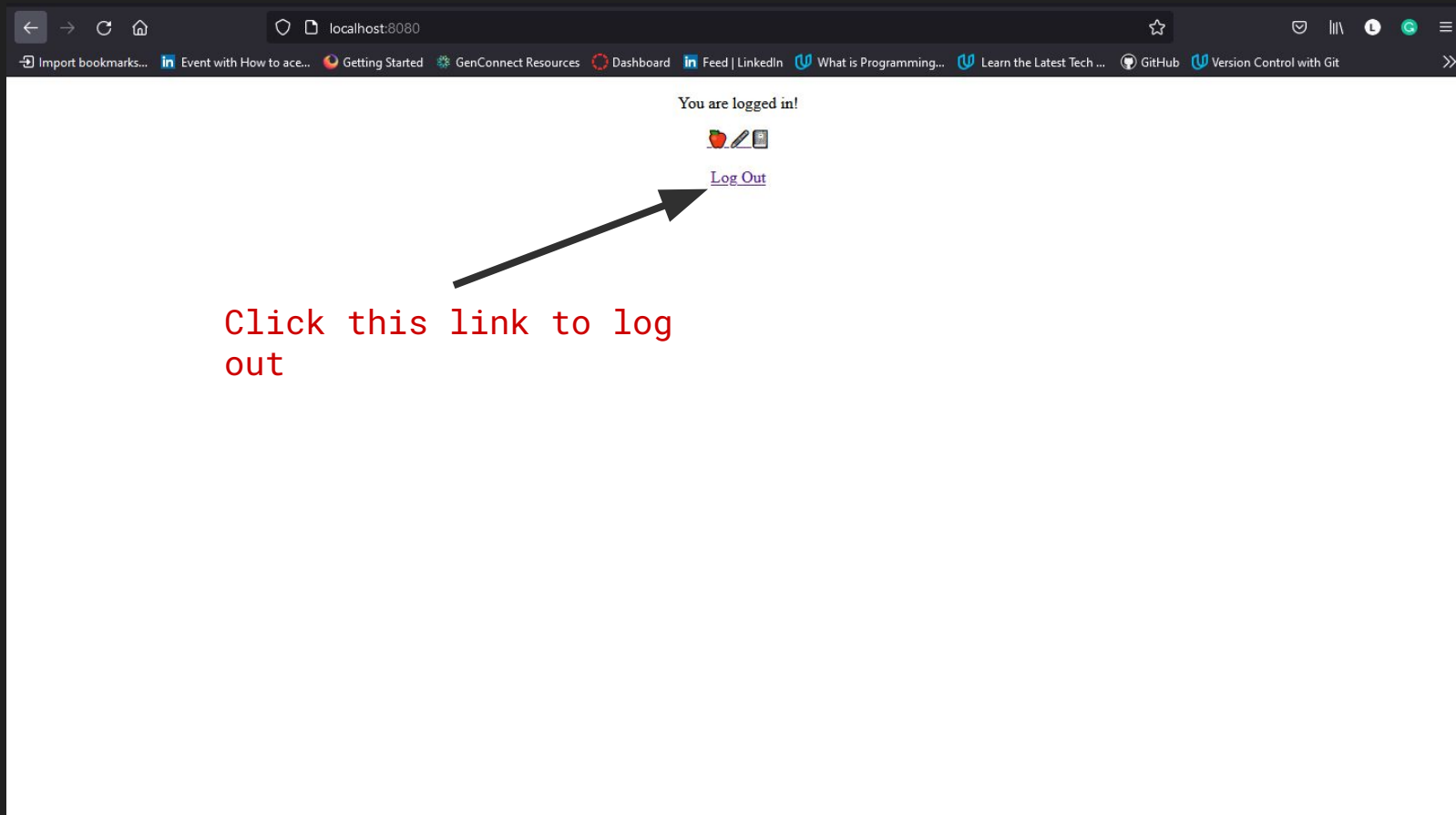


<http://localhost:8080/api/criminals> GET

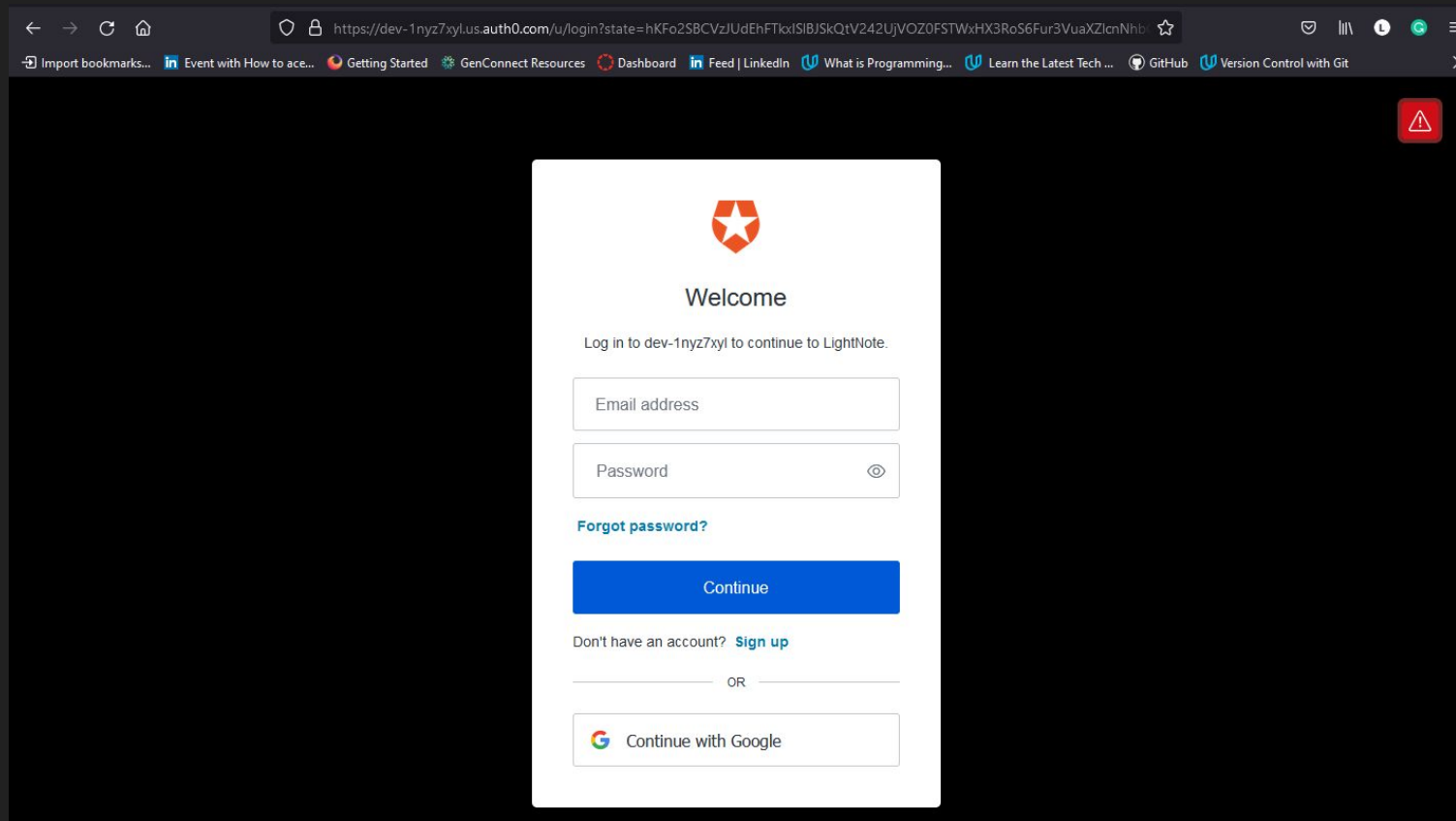
As you can see Speedy is no longer marked for death



Once authenticated you are logged in



After logging out you are returned to the oauth page



A screenshot of a web browser displaying the login page for 'dev-1nyz7xyl.us.auth0.com'. The browser's address bar shows the URL, and the bookmarks bar lists various links. The login form is centered on a dark background and includes a logo, a welcome message, login instructions, input fields for email and password, a 'Continue' button, a 'Forgot password?' link, a 'Sign up' link, and a 'Continue with Google' button.

← → ↻ 🏠 https://dev-1nyz7xyl.us.auth0.com/u/login?state=hKFo2SBCVzJUdEhFTkdSIBJSkQtV242UjVOZ0FSTWxHX3RoS6Fur3VuaXZlcnNhbi... ☆

🔖 Import bookmarks... 🌐 Event with How to ace... 📺 Getting Started 🌐 GenConnect Resources 📺 Dashboard 🌐 Feed | LinkedIn 🗨️ What is Programming... 🗨️ Learn the Latest Tech ... 🗨️ GitHub 🗨️ Version Control with Git >

⚠️



Welcome

Log in to dev-1nyz7xyl to continue to LightNote.

Email address

Password 

[Forgot password?](#)

Continue

Don't have an account? [Sign up](#)

OR

 Continue with Google

Segment 1 – RESTful Design

```
@RestController // "Every request handling method of the controller class automatically serializes return objects into
  HttpServletResponse"
@RequestMapping("/api/criminals") // Used to map web requests to Spring Controller methods
public class CriminalController {
```

```
    @PostMapping // Annotation for mapping HTTP POST requests onto specific handler methods
    public Criminal createCriminal(@RequestBody Criminal criminal) {
        return this.criminalService.save(criminal);
    }
```

```
    // build get user by id REST API
    @GetMapping("/{id}") // Annotation for mapping HTTP GET requests onto specific handler methods
    public ResponseEntity<Criminal> getCriminalById(@PathVariable long id){
        Criminal criminal = criminalRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Criminal does not exist with id:" + id));
        return ResponseEntity.ok(criminal);
    }
```

```
    // build update user REST API
    @PutMapping("/{id}") // Annotation for mapping HTTP PUT requests onto specific handler methods
    public ResponseEntity<Criminal> updateCriminal(@PathVariable long id, @RequestBody Criminal criminalDetails) {
        // @PathVariable annotation to extract the templated part of the URI (id), @RequestBody maps the HttpRequest body to a
        // transfer or domain object, enabling automatic deserialization of the inbound HttpRequest body onto a Java object
        Criminal updateCriminal = criminalRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Criminal does not exist with id: " + id));
```

```
        updateCriminal.setName(criminalDetails.getName());
        updateCriminal.setMod(criminalDetails.getMod());
        updateCriminal.setImageLink(criminalDetails.getImageLink());

        criminalRepository.save(updateCriminal);

        return ResponseEntity.ok(updateCriminal);
    }
```

Paths represent “nouns” or
resources, HTTP “verbs” map
to data operations.

```
    // build delete user REST API
    @DeleteMapping("/{id}") // Annotation for mapping HTTP DELETE requests onto specific handler methods.
    public ResponseEntity<HttpStatus> deleteCriminal(@PathVariable long id){

        Criminal criminal = criminalRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Criminal does not exist with id: " + id));

        criminalRepository.delete(criminal);

        return new ResponseEntity<>(HttpStatus.NO_CONTENT);
    }
```

GET	/criminals	Show all criminals
GET	/criminals/3	Show a single criminal (whose ID=3 in the DB)
POST	/criminals	Create a new criminal in the DB
PUT	/criminals/3	Update user 3 in the DB
DELETE	/criminals/3	Delete user 3 from the DB

Segment 2 – Auth

We implemented OAuth
SSO (single sign-on)



Welcome

Log in to dev-1nyz7xyl to continue to LightNote.

Email address

Password



[Forgot password?](#)

Continue

Don't have an account? [Sign up](#)

OR



Continue with Google

AuthController.java

```
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.core.oidc.user.OidcUser;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

/**
 * Controller for the home page.
 */
@Controller
public class AuthController {

    @GetMapping("/")
    public String home(Model model, @AuthenticationPrincipal OidcUser
principal) { return "index"; }
}
```

SecurityConfig.java

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.authorizeRequests().anyRequest().authenticated
() ExpressionUrlAuthorizationConfigurer<...>.ExpressionInterceptUrlRegistry
    .and() HttpSecurity
    .oauth2Login() OAuth2LoginConfigurer<HttpSecurity>
    .and().logout() LogoutConfigurer<HttpSecurity>
    .logoutRequestMatcher(new AntPathRequestMatcher
( pattern: "/logout"))
    .addLogoutHandler(LogoutHandler)
//    ** IMPORTANT! do not use the line below in production apps!! **
    .and().csrf().disable() HttpSecurity
    .cors() CorsConfigurer<HttpSecurity>
    .and().oauth2ResourceServer().jwt();
}
```

LogoutHandler.java

```
/**
 * Needed to perform SSO logout with Auth0. By default, Spring will clear the
 * SecurityContext and the session.
 * This controller will also log users out of Auth0 by calling the Auth0 logout
 * endpoint.
 */
@Controller
public class LogoutHandler extends SecurityContextLogoutHandler {

    private final ClientRegistrationRepository clientRegistrationRepository;

    /**
     * Create a new instance with a {@code ClientRegistrationRepository}, so
     * that we can look up information about the
     * configured provider to call the Auth0 logout endpoint. Called by the
     * Spring framework.
     *
     * @param clientRegistrationRepository the {@code ClientRegistrationRepository} for this application.
     */
    @Autowired
    public LogoutHandler(ClientRegistrationRepository
clientRegistrationRepository) {
        this.clientRegistrationRepository = clientRegistrationRepository;
    }

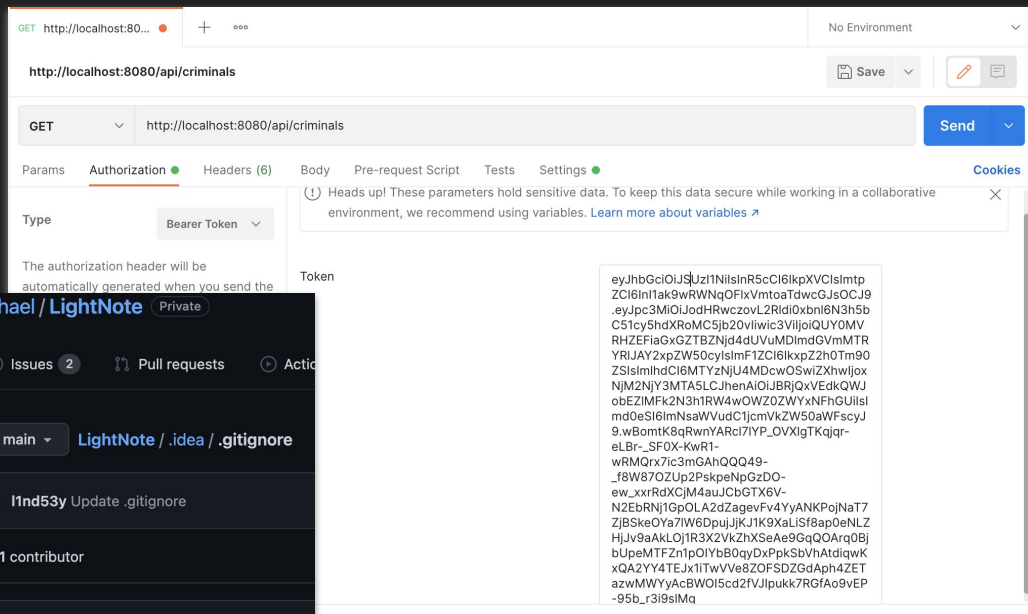
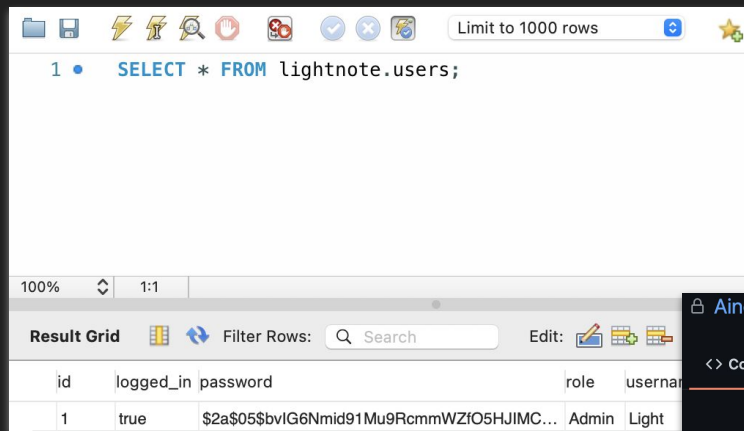
    /**
     * Delegates to {@linkplain SecurityContextLogoutHandler} to log the user
     * out of the application, and then logs
     * the user out of Auth0.
     */
}
```

Given that only Light and his
“disciples” should have access to the
DB, we hoped to be able to also
configure permissions, but ran into
time constraints and bugs.

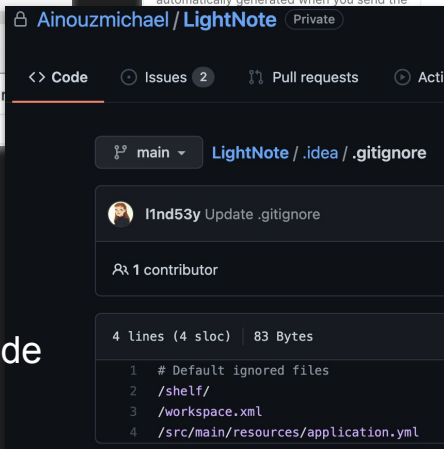
Segment 3 – Testing + Security

Implemented hashing on the user password using Bcrypt.

Protected our API by requiring a Bearer token to perform any CRUD operations.



We put our application.yml file inside our .gitignore file.



Segment 4 – Agile

- Project management / git: We found it helpful to agree not to push to main or merge branches without team members' approval, daily standups, pair programming
- Learning moment: We gained a lot of experience with productive struggling, independent troubleshooting, and git issues
- What we would do differently: Planning ahead for the project by studying resources more before starting work on the project (Java, Spring, Authentication/Authorization, etc.)

THE END. Thank you!

